STM – DOCKER E KUBERNETES

Instrutoria Interna

Deployments, Services, ConfigMaps, Secrets e Rollouts

Gerenciamento Avançado de Workloads, Redes e Configurações Externa no Kubernetes

Wescley de Macêdo Sousa
wescleysousa@stm.jus.br

DEPLOYMENTS, SERVICES, CONFIGMAPS E SECRETS NO KUBERNETES

SUMÁRIO

MÓDULO 1: GERENCIAMENTO DECLARATIVO DE CARGAS DE TRABALHO (DEPLOYMENTS E REPLICASETS) 3

1.1 A NECESSIDADE DE ABSTRAÇÃO SOBRE PODS AVULSOS 3

1.2 A HIERARQUIA DE RECURSOS: DEPLOYMENT REPLICASET POD 3

1.3 A FILOSOFIA DECLARATIVA E O MECANISMO DE AUTOCURA (SELF-HEALING) 5

1.4 ANATOMIA DETALHADA DE UM MANIFESTO deployment.yaml 6

MÓDULO 2: DESCOBERTA DE SERVIÇOS E COMUNICAÇÃO DE REDE (SERVICES) 7

2.1 O PROBLEMA DA VOLATILIDADE DOS IPS DOS PODS 7

2.2 O OBJETO SERVICE E A ABSTRAÇÃO DE ENDEREÇAMENTO ESTÁVEL 7

2.3 SELETORES DE RÓTULOS (LABEL SELECTORS) E ENDPOINTS 8

2.4 TIPOS DE SERVICES 8

2.5 RESOLUÇÃO DNS INTERNA (COREDNS) E COMUNICAÇÃO ENTRE APLICAÇÕES 9

MÓDULO 3: EXTERNALIZANDO CONFIGURAÇÕES E DADOS SENSÍVEIS (CONFIGMAPS E SECRETS) 10

3.1 O PRINCÍPIO DOS 12-FACTOR APPS 10

3.2 CONFIGMAPS: CONCEITO E ANATOMIA 10

3.3 SECRETS: CONCEITO, TIPOS E SEGURANÇA 11

3.4 ESTRATÉGIAS DE INJEÇÃO EM PODS 13

MÓDULO 4: ESCALABILIDADE E GERENCIAMENTO DO CICLO DE VIDA (ROLLOUTS E ROLLBACKS) 16

4.1 ESCALABILIDADE DE CARGAS DE TRABALHO 16

4.2 ESTRATÉGIAS DE ATUALIZAÇÃO DE APLICAÇÕES 16

4.3 O MECANISMO DE ROLLOUT STATUS, HISTÓRICO E REVERSÃO (ROLLBACK) 17

4.4 ORGANIZAÇÃO E ISOLAMENTO AVANÇADO COM NAMESPACES 18

MÓDULO 5: LABORATÓRIO PRÁTICO INTEGRADO (IMPLANTAÇÃO DE UMA APLICAÇÃO COMPLETA) 19

MÓDULO 6: GUIA DE CONSULTA RÁPIDA (CHEATSHEET DE COMANDOS) 31

REFERÊNCIAS 33

MÓDULO 1: GERENCIAMENTO DECLARATIVO DE CARGAS DE TRABALHO (DEPLOYMENTS E REPLICASETS)

1.1 A NECESSIDADE DE ABSTRAÇÃO SOBRE PODS AVULSOS

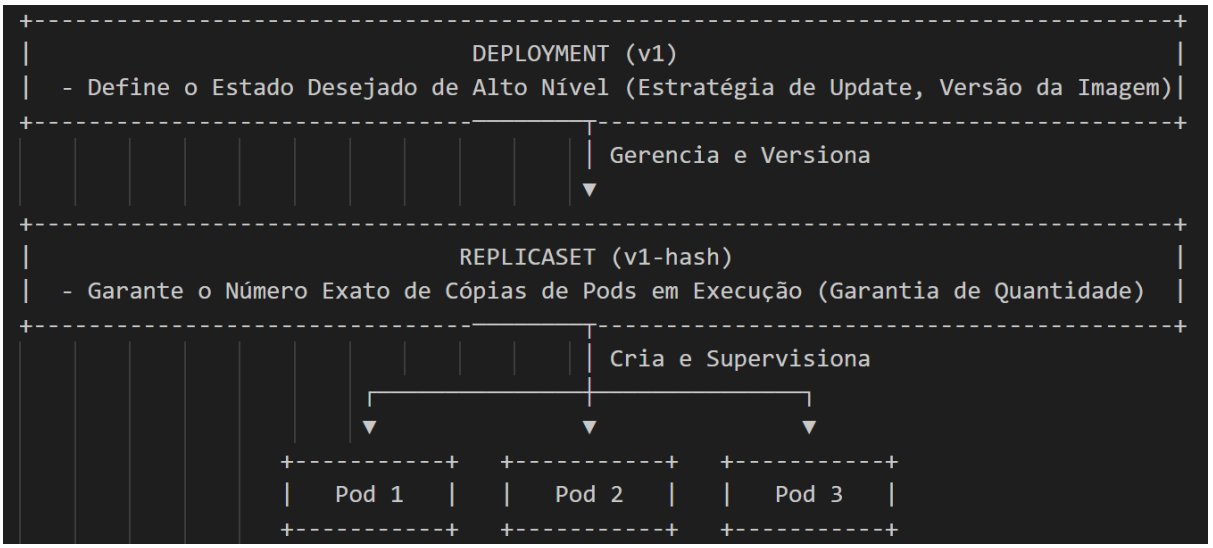
No módulo anterior, analisamos a estrutura e a criação de Pods individuais (Pods *ad-hoc* ou "nativos"). Embora o Pod seja a menor unidade computacional no Kubernetes, gerenciar cargas de trabalho diretamente por Pods manuais apresenta sérias limitações para ambientes de produção:

1. **Inexistência de Garantia de Resiliência:** Se o nó hospedeiro onde um Pod avulso está rodando falhar fisicamente, o Pod é destruído e **não é recriado automaticamente** em outro nó.
2. **Incapacidade de Escalonamento Automático:** Para aumentar o número de instâncias de uma aplicação baseada em Pods avulsos, o operador precisaria criar manualmente novos manifestos com nomes distintos para cada Pod.
3. **Impossibilidade de Atualização Sem Parada (*Zero-Downtime Updates*):** Atualizar a versão da imagem de um Pod exige deletá-lo e recriá-lo, gerando uma janela inevitável de indisponibilidade (*downtime*).

Para resolver esses problemas, o Kubernetes adota o padrão de **Controladores de Carga de Trabalho (*Workload Controllers*)**, sendo o **Deployment** o mais utilizado para aplicações sem estado (*stateless*).

1.2 A HIERARQUIA DE RECURSOS: DEPLOYMENT REPLICASET POD

O gerenciamento de aplicações no Kubernetes não é feito por um único bloco monolítico, mas por uma arquitetura hierárquica e desacoplada em três níveis:



Nível 1: O Deployment

O **Deployment** é o objeto de mais alto nível. Ele não gerencia os Pods diretamente; em vez disso, gerencia os **ReplicaSets**. O Deployment é responsável por:

- Definir a especificação do Pod (*PodTemplate*).
- Gerenciar atualizações de versão da aplicação (*Rollouts*).
- Manter o histórico de revisões para permitir reversões (*Rollbacks*).

Nível 2: O ReplicaSet

O **ReplicaSet** é um controlador secundário cujo único propósito é garantir que um número específico de instâncias de Pods idênticos esteja em execução a qualquer momento.

- Se um Pod cai, o ReplicaSet detecta e cria um novo.
- Se surgem Pods em excesso (por exemplo, criados manualmente com as mesmas etiquetas), o ReplicaSet remove os excedentes.
- Quando alteramos a versão da imagem em um Deployment, o Deployment **cria um novo ReplicaSet** para a nova versão e reduz gradualmente o ReplicaSet antigo a zero.

Nível 3: O Pod

A unidade final de execução que armazena os containers e roda no nó Worker.

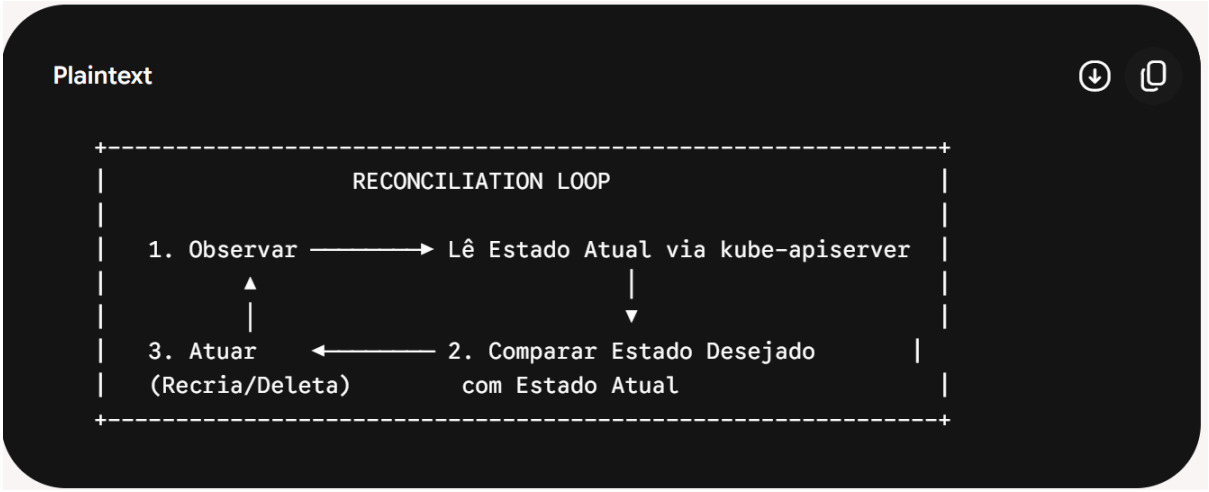
1.3 A FILOSOFIA DECLARATIVA E O MECANISMO DE AUTOCURA (SELF-HEALING)

O Kubernetes opera sob o paradigma **Declarativo**. Em vez de instruir o cluster sobre *como* fazer algo (paradigma imperativo), o operador declara *qual deve ser o estado final do cluster* no manifesto YAML.

O **Loop de Reconciliação (*Reconciliation Loop*)** do Kubernetes monitora continuamente dois estados:

Estado Desejado (Desired State) <===> Estado Atual (Current State)

Se um evento faz com que o *Estado Atual* divirja do *Estado Desejado*, o controlador atua imediatamente para corrigir a discrepância:



Exemplo Prático de Autocura (*Self-Healing*):

1. O operador declara um Deployment com `replicas: 3`.
2. O ReplicaSet cria os Pods `A`, `B` e `C`.
3. O nó onde o Pod `B` está rodando sofre uma falha de hardware e desliga.
4. O `kubelet` para de enviar *heartbeats* ao Control Plane.
5. O ReplicaSet percebe que agora exibe apenas 2 Pods ativos (*Current State = 2*), enquanto a especificação exige 3 (*Desired State = 3*).
6. O ReplicaSet solicita imediatamente ao `kube-apiserver` a criação do Pod `D` em um nó ativo e saudável.

1.4 ANATOMIA DETALHADA DE UM MANIFESTO `deployment.yaml`

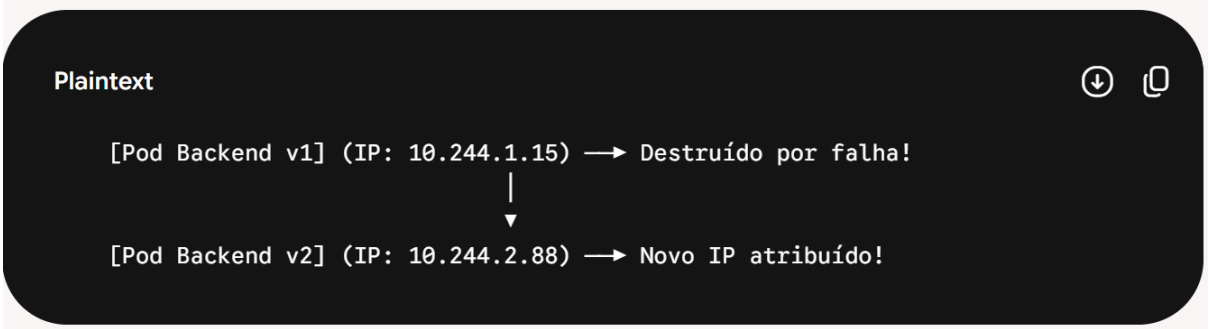
Abaixo está a estrutura completa e comentada de um manifesto de Deployment para uma API Backend:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-backend-deployment
  namespace: default
  labels:
    app: api-backend
    tier: backend
spec:
  replicas: 3                     # Quantidade desejada de instâncias de Pods
  selector:
    matchLabels:
      app: api-backend            # Filtro para o ReplicaSet identificar seus Pods
  strategy:
    type: RollingUpdate           # Estratégia de atualização padrão
    rollingUpdate:
      maxSurge: 1                 # Quantos Pods acima do especificado podem ser criados
      maxUnavailable: 0           # Quantos Pods podem ficar indisponíveis durante o update
  template:                       # Modelo (PodTemplate) utilizado para criar os Pods
    metadata:
      labels:
        app: api-backend          # DEVE corresponder estritamente ao matchLabels acima!
        tier: backend
    spec:
      containers:
        - name: container-api
          image: nginx:1.25-alpine # Imagem da aplicação
          ports:
            - containerPort: 80    # Porta interna exposta pelo container
          resources:
            requests:
              cpu: "100m"          # Reserva mínima de 0.1 CPU
              memory: "128Mi"      # Reserva mínima de 128 MB de RAM
            limits:
              cpu: "500m"          # Limite máximo de 0.5 CPU
              memory: "256Mi"      # Limite máximo de 256 MB de RAM
```

MÓDULO 2: DESCOBERTA DE SERVIÇOS E COMUNICAÇÃO DE REDE (SERVICES)

2.1 O PROBLEMA DA VOLATILIDADE DOS IPS DOS PODS

No Kubernetes, os Pods são efêmeros e descartáveis. Toda vez que um Pod é destruído e recriado pelo mecanismo de autocura do ReplicaSet, ou quando o Deployment é atualizado, **o novo Pod recebe um endereço IP completamente novo**.

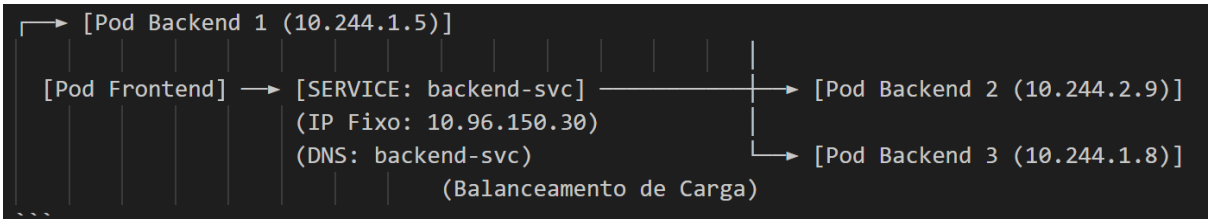


Se o Frontend tentasse se conectar diretamente ao endereço IP `10.244.1.15`, a comunicação falharia assim que o Pod fosse substituído. Para resolver essa instabilidade de rede, o Kubernetes introduz o objeto **Service**.

2.2 O OBJETO SERVICE E A ABSTRAÇÃO DE ENDEREÇAMENTO ESTÁVEL

Um **Service** é uma abstração que define um conjunto lógico de Pods e uma política de acesso a eles. O Service fornece:

1. Um **Endereço IP Virtual Fixo (ClusterIP)** que nunca muda durante o ciclo de vida do serviço.
2. Um **Nome DNS Estável** registrado no servidor DNS interno do cluster (CoreDNS).
3. **Balanceamento de Carga Automático (L4)** distribuindo as requisições entre os Pods saudáveis no backend.



2.3 SELETORES DE RÓTULOS (LABEL SELECTORS) E ENDPOINTS

A conexão entre um Service e seus Pods correspondentes não é feita por nomes explícitos, mas sim por **Rótulos (*Labels*)** e **Seletores (*Selectors*)**.

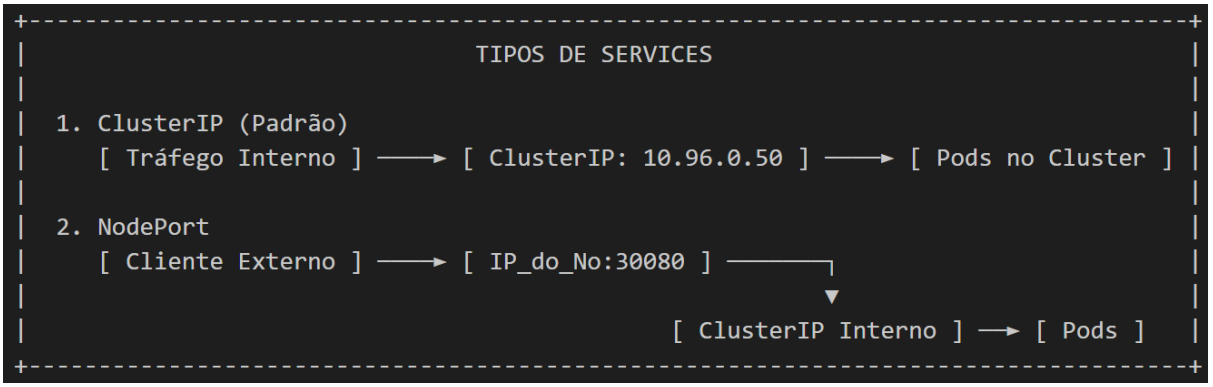
O Kubernetes cria automaticamente um objeto auxiliar chamado **Endpoints** (ou *EndpointSlices*), que armazena a lista em tempo real dos endereços IP de todos os Pods que correspondem às labels definidas no Service.

```
# Trecho do Service
spec:
  selector:
    app: api-backend  # Procura por qualquer Pod que possua a label 'app: api-backend'
```

Se o `healthcheck` de um Pod falhar, o Kubernetes o remove imediatamente do objeto *Endpoints*, impedindo que o Service envie tráfego para uma instância doente.

2.4 TIPOS DE SERVICES

O Kubernetes oferece diferentes tipos de Services para atender a necessidades distintas de exposição de rede:



1. ClusterIP (Padrão)

- Expõe o Service em um **IP interno** acessível **apenas dentro do cluster**.
- Utilizado para comunicação entre componentes internos (ex: Frontend acessando Backend, ou Backend acessando o Banco de Dados).
- Não é acessível pela internet ou por redes externas ao cluster.

2. NodePort

- Constrói sobre o ClusterIP, abrindo uma **porta estática em cada Nó do Cluster** (faixa padrão: 30000 a 32767).
- Permite que tráfego vindo de fora do cluster acesse o Service através do IP de qualquer nó na porta atribuída: <IP-do-No>:<NodePort>.
- Utilizado para ambientes de desenvolvimento, testes ou quando não há um Load Balancer de nuvem integrado.

3. LoadBalancer

- Integra-se com APIs de provedores de nuvem (AWS, GCP, Azure) para provisionar automaticamente um Load Balancer externo da nuvem (ex: AWS ALB/NLB) direcionando o tráfego para os nós.

4. ExternalName

- Mapeia o Service para um nome DNS externo (ex: meu-banco.rds.amazonaws.com), permitindo que aplicações internas acessem serviços externos usando a resolução DNS do cluster.

2.5 RESOLUÇÃO DNS INTERNA (COREDNS) E COMUNICAÇÃO ENTRE APLICAÇÕES

O Kubernetes executa internamente o serviço **CoreDNS**. Qualquer Service criado no cluster ganha automaticamente uma entrada DNS completa no seguinte formato:

$$< nome - do - servico >.< namespace >.svc.cluster.local$$

Padrões de Comunicação via DNS:

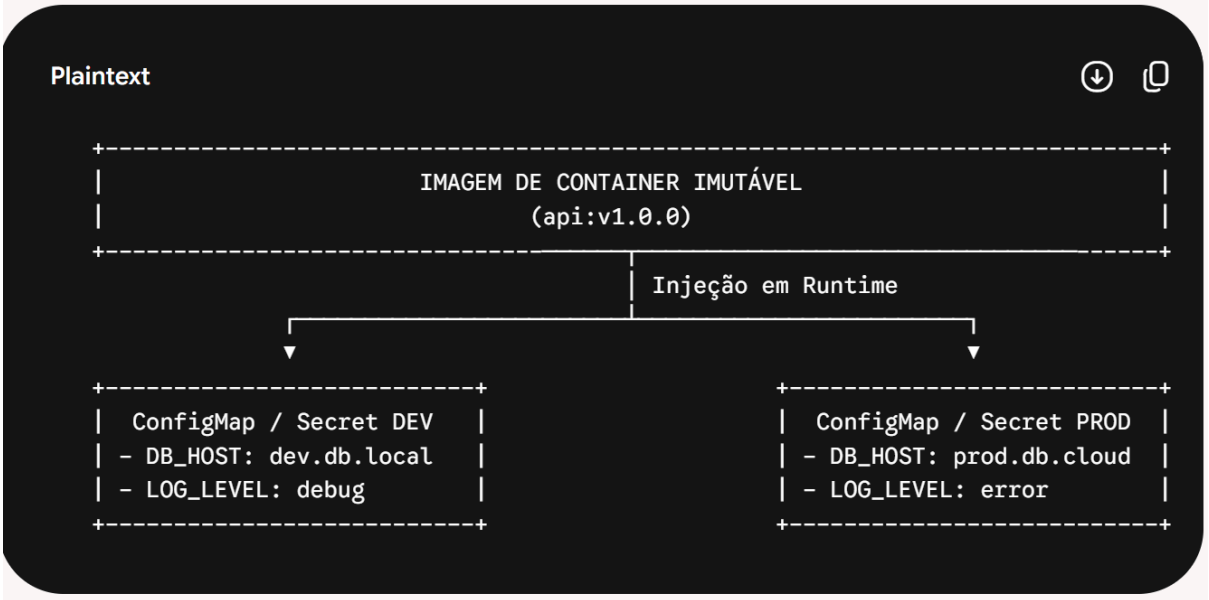
- **Mesmo Namespace:** A aplicação pode fazer requisições chamando apenas o nome do serviço: http://backend-svc:8080.
- **Namespaces Diferentes:** A aplicação deve explicitar o namespace de destino: http://backend-svc.desenvolvimento:8080.

MÓDULO 3: EXTERNALIZANDO CONFIGURAÇÕES E DADOS SENSÍVEIS (CONFIGMAPS E SECRETS)

3.1 O PRINCÍPIO DOS 12-FACTOR APPS

A metodologia dos **12-Factor Apps (Aplicações de Doze Fatores)** estipula rigorosamente no seu Fator III: **Configuração (Config)** que o código de uma aplicação deve ser completamente agnóstico do ambiente em que executa.

> **Regra de Ouro:** *O código da aplicação compila uma única imagem imutável de container. Todas as configurações que mudam entre ambientes (Desenvolvimento, Homologação, Produção) devem ser* **externalizadas** *e injetadas em tempo de execução.*



3.2 CONFIGMAPS: CONCEITO E ANATOMIA

Um **ConfigMap** é um objeto Kubernetes utilizado para armazenar dados não confidenciais em pares chave-valor ou na forma de arquivos de configuração completos.

Exemplo de Manifesto `configmap.yaml`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
data:
  # Par chave-valor simples
  LOG_LEVEL: "info"
  PORTA_APLICACAO: "8080"

  # Arquivo de configuração estruturado
  app-settings.json: |
    {
      "cacheEnabled": true,
      "timeoutMs": 5000,
      "maxConnections": 100
    }
```

3.3 SECRETS: CONCEITO, TIPOS E SEGURANÇA

O objeto **Secret** destina-se ao armazenamento de informações sensíveis, tais como senhas, tokens OAuth, chaves de API e certificados TLS.

Tipos Nativos de Secrets:

1. **`Opaque`:** O tipo padrão para pares chave-valor arbitrários definidos pelo usuário.
2. **`kubernetes.io/service-account-token`:** Armazena tokens de autenticação para contas de serviço no cluster.
3. **`kubernetes.io/dockerconfigjson`:** Armazena credenciais de autenticação para registries privados de imagens Docker.
4. **`kubernetes.io/tls`:** Armazena certificados TLS (chave privada e certificado público) para terminação HTTPS.

A Codificação Base64 e o Mito da Segurança

Por padrão, os dados em um arquivo de manifesto YAML de Secret são salvos codificados em **Base64**.

> ⚠️ ATENÇÃO TÉCNICA: Base64 NÃO É CRIPTOGRAFIA!
> *Base64 é apenas um formato de codificação de texto (encoding). Qualquer pessoa com acesso de leitura ao manifesto pode decodificar o valor instantaneamente executando:* `echo "c2VuaGFfc2VjcmV0YQ==" | base64 --decode`.

Bash

```bash
# Como codificar uma senha em Base64 no Linux/macOS:
echo -n "minha_senha_super_segura" | base64
# Resultado: bWluaGFfc2VuaGFfc3VwZXJfc2VndXJh
```

Exemplo de Manifesto `secret.yaml`:

YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: default
type: Opaque
data:
  # Os valores DEVEM obrigatoriamente estar em Base64
  DB_PASSWORD: bWluaGFfc2VuaGFfc3VwZXJfc2VndXJh
  API_TOKEN: dG9rZW5fYXBpX3NlY3JldG9fMTIzNA==
```

Boas Práticas de Segurança para Secrets em Produção:

- **Ativar Criptografia em Repouso (*Encryption at Rest*):** Configurar o `kube-apiserver` para criptografar dados do `etcd` usando chaves de criptografia (ex: AES-GCM ou KMS de nuvem).
- **Controle de Acesso estrito via RBAC:** Restringir quais usuários e ServiceAccounts podem ler objetos `secrets`.
- **Utilizar Gestores Externos de Secrets:** Em ambientes corporativos, adote ferramentas como *HashiCorp Vault*, *AWS Secrets Manager* ou *Azure Key Vault* integrados via **External Secrets Operator (ESO)**.

3.4 ESTRATÉGIAS DE INJEÇÃO EM PODS

O Kubernetes oferece duas estratégias fundamentais para disponibilizar os dados de ConfigMaps e Secrets dentro dos containers:

Estratégia 1: Injeção como Variáveis de Ambiente

Os valores das chaves são passados diretamente como variáveis de ambiente para o processo do container.

```yaml
# Injeção no container via env/envFrom
spec:
  containers:
    - name: api
      image: nginx:alpine
      env:
        # Injetando uma chave específica de um ConfigMap
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: LOG_LEVEL
        # Injetando uma chave específica de um Secret
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: DB_PASSWORD
```

Ou injetando **todas** as chaves de uma só vez:

```yaml
      envFrom:
        - configMapRef:
            name: app-config
        - secretRef:
            name: app-secrets
```

Estratégia 2: Montagem como Volumes do Sistema de Arquivos (*Volume Mounts*)

Os dados do ConfigMap ou Secret são montados dentro do sistema de arquivos do container como arquivos individuais dentro de um diretório.

```yaml
spec:
  containers:
    - name: api
      image: nginx:alpine
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config     # Cria o diretório /etc/config no container
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: app-config             # Cada chave do ConfigMap vira um arquivo dentro de /etc/config
```

Comparativo entre Variáveis de Ambiente e Volumes:

Característica	Variáveis de Ambiente	Montagem por Volume (*Volume Mount*)
Complexidade de Uso	Muito simples de ler no código.	Requer leitura de arquivo no disco.
Atualização Dinâmica	**Estática.** Exige reinicialização (*restart*) do Pod para refletir novos valores.	**Dinâmica.** Alterações no ConfigMap/Secret atualizam os arquivos no disco em poucos segundos sem reiniciar o Pod.
Suporte a Arquivos Grandes	Inadequado para arquivos estruturados.	**Ideal** para arquivos JSON, YAML, Nginx config ou certificados TLS.

MÓDULO 4: ESCALABILIDADE E GERENCIAMENTO DO CICLO DE VIDA (ROLLOUTS E ROLLBACKS)

4.1 ESCALABILIDADE DE CARGAS DE TRABALHO

A escalabilidade em sistemas distribuídos pode ocorrer de forma manual ou automatizada.

Escalabilidade Manual

O operador altera explicitamente a quantidade de réplicas desejadas no controlador. Isso pode ser feito alterando o arquivo YAML e executando `kubectl apply -f`, ou diretamente via CLI:

```bash
# Redimensiona o Deployment para 5 réplicas
kubectl scale deployment api-backend-deployment --replicas=5
```

Escalabilidade Automática (HPA)

Em ambientes avançados, utiliza-se o objeto **Horizontal Pod Autoscaler (HPA)**, que ajusta automaticamente o número de réplicas com base na utilização real de CPU, memória ou métricas customizadas de requisições por segundo.

4.2 ESTRATÉGIAS DE ATUALIZAÇÃO DE APLICAÇÕES

Quando modificamos a especificação da imagem em um Deployment, o Kubernetes precisa atualizar os Pods em execução. A propriedade `spec.strategy.type` define como essa transição ocorre:

```
1. Estratégia "Recreate" (Com Downtime):
   [ Pods v1 (3) ] ──► [ Todos Deletados (0) ] ──► [ Janela Indisponível ] ──► [ Pods v2 (3) ]

2. Estratégia "RollingUpdate" (Sem Downtime - Padrão):
   [ Pods v1 (3) ] ──► [ Cria Pod v2 (1) ] ──► [ Destrói Pod v1 (1) ] ──► ... ──► [ Pods v2 (3) ]
```

Estratégia 1: `Recreate`

- Todos os Pods da versão antiga são finalizados e deletados de uma só vez.
- O cluster aguarda a destruição total e só então cria os Pods da nova versão.
- **Gera janela de indisponibilidade (*Downtime*)**, mas evita que duas versões diferentes da aplicação rodem simultaneamente (útil para migrações breaking de banco de dados).

Estratégia 2: `RollingUpdate` (Padrão)

- O Deployment atualiza os Pods gradualmente, criando instâncias da nova versão enquanto reduz proporcionalmente as instâncias antigas.
- **Garante zero downtime**, pois sempre haverá Pods ativos servindo tráfego durante todo o processo.

Os Parâmetros de Controle do RollingUpdate:

- **`maxSurge`:** Define o número (ou porcentagem) máximo de Pods que podem ser criados **acima** da quantidade de réplicas estipulada durante o update.
- **`maxUnavailable`:** Defines o número (ou porcentagem) máximo de Pods que podem ficar **indisponíveis** durante o processo de atualização.

4.3 O MECANISMO DE ROLLOUT STATUS, HISTÓRICO E REVERSÃO (ROLLBACK)

Toda alteração efetuada na especificação do modelo de Pod (*PodTemplate*) de um Deployment gera uma nova **Revisão (*Revision*)** mantida pelo histórico de ReplicaSets.

Fluxo do Histórico e Reversão:

1. **Verificar o Status do Rollout:**

```bash
kubectl rollout status deployment/api-backend-deployment
```

2. Visualizar o Histórico de Revisões:

```bash
kubectl rollout history deployment/api-backend-deployment
```

3. Executar a Reversão (*Rollback*) em caso de falha:

Se uma nova versão com erro for implantada (ex: imagem corrompida), o comando undo reverte o ambiente instantaneamente para a revisão estável anterior:

```bash
kubectl rollout undo deployment/api-backend-deployment
```

4.4 ORGANIZAÇÃO E ISOLAMENTO AVANÇADO COM NAMESPACES

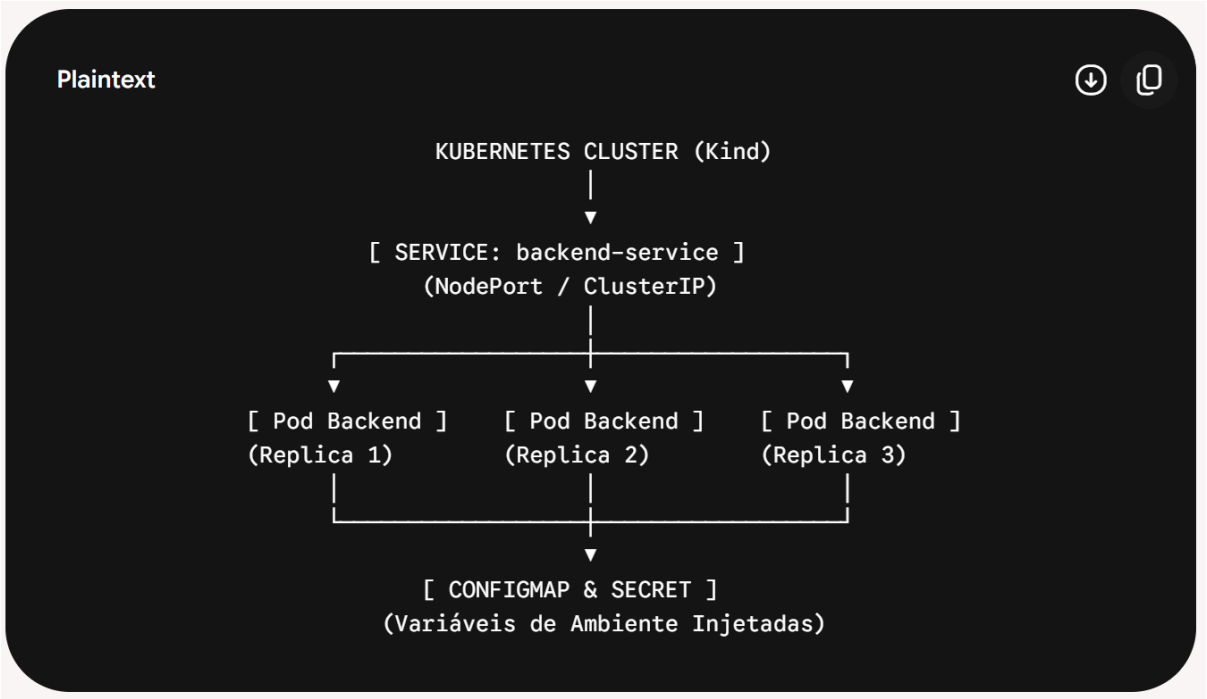
Para gerenciar múltiplos ambientes e equipes em um mesmo cluster, utilizamos **Namespaces**.

```bash
# Criar um namespace para isolar o ambiente de treinamento
kubectl create namespace treinamento
```

Ao aplicar manifestos informando a flag `-n <namespace>` ou declarando `metadata.namespace` nos YAMLs, garantimos o isolamento lógico das cargas de trabalho, cuotas de recursos (*ResourceQuotas*) e políticas de segurança (*NetworkPolicies*).

MÓDULO 5: LABORATÓRIO PRÁTICO INTEGRADO (IMPLANTAÇÃO DE UMA APLICAÇÃO COMPLETA)

Neste laboratório prático, construiremos a arquitetura distribuída completa solicitada, unindo todos os conceitos explorados no curso.



Passo 1: Estrutura dos Arquivos do Laboratório

Crie um diretório de trabalho na sua máquina e monte os 4 manifestos declarativos a seguir:



Passo 2: O ConfigMap (`01-configmap.yaml`)

Crie o arquivo **`01-configmap.yaml`**:

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: default
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  WELCOME_MSG: "Bem-vindo a API orquestrada pelo Kubernetes!"
```

Passo 3: O Secret (02-secret.yaml)

Crie o arquivo **02-secret.yaml**. Note que os valores estão codificados em Base64:

- admin_user em Base64 → YWRtaW5fdXNlcg==
- senha_mestra_123 em Base64 → c2VuaGFfbWVzdHJhXzEyMw==

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: backend-secret
  namespace: default
type: Opaque
data:
  DB_USER: YWRtaW5fdXNlcg==
  DB_PASS: c2VuaGFfbWVzdHJhXzEyMw==
```

Passo 4: O Deployment Integrado (`03-deployment.yaml`)

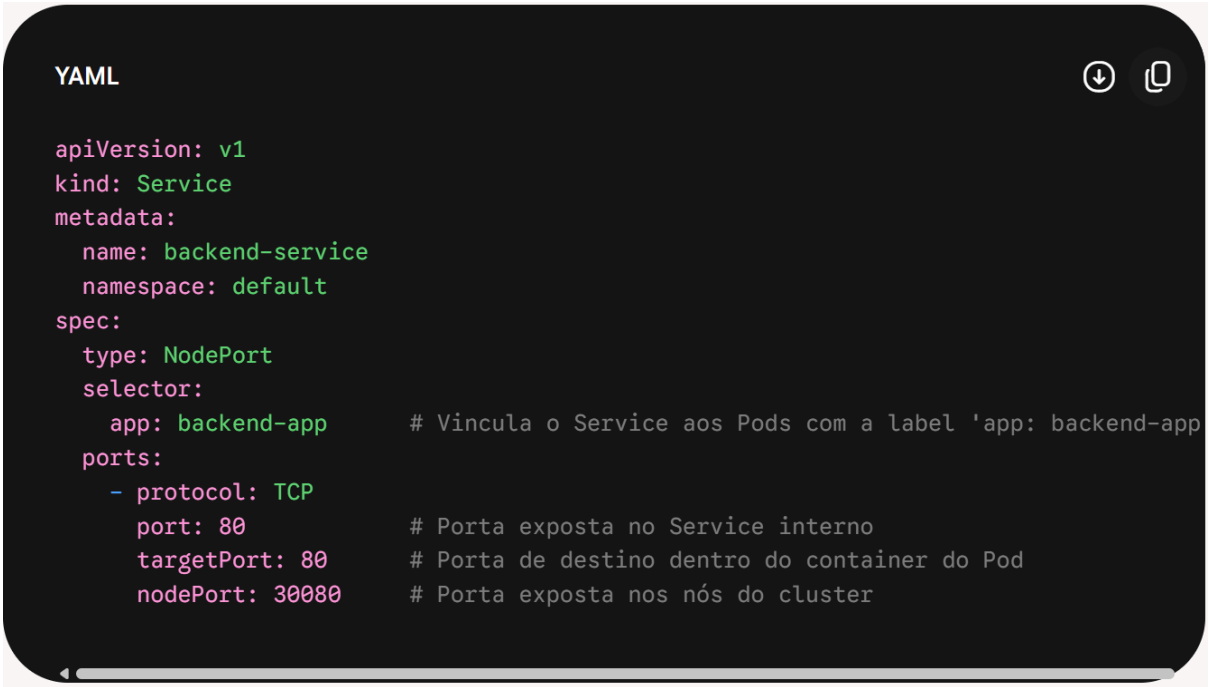
Crie o arquivo **`03-deployment.yaml`**. Este manifesto declara 3 réplicas, injeta o ConfigMap e o Secret como variáveis de ambiente e define checagens de recursos.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: default
  labels:
    app: backend-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: backend-app
    spec:
      containers:
        - name: container-backend
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: backend-config
            - secretRef:
                name: backend-secret
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "250m"
              memory: "128Mi"
```

Passo 5: O Service (`04-service.yaml`)

Crie o arquivo **`04-service.yaml`**. Utilizaremos o tipo `NodePort` para permitir o acesso direto à aplicação.



```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: default
spec:
  type: NodePort
  selector:
    app: backend-app      # Vincula o Service aos Pods com a label 'app: backend-app
  ports:
    - protocol: TCP
      port: 80            # Porta exposta no Service interno
      targetPort: 80      # Porta de destino dentro do container do Pod
      nodePort: 30080     # Porta exposta nos nós do cluster
```

Passo 6: Aplicação e Execução da Atividade Prática

Siga a sequência estrita de comandos no terminal para implantar os objetos e executar o ciclo completo da atividade prática solicitada.

1. Aplicar todos os Manifestos YAML:

```bash
kubectl apply -f 01-configmap.yaml
kubectl apply -f 02-secret.yaml
kubectl apply -f 03-deployment.yaml
kubectl apply -f 04-service.yaml
```

Saída Esperada:

```
configmap/backend-config created
secret/backend-secret created
deployment.apps/backend-deployment created
service/backend-service created
```

2. Consultar a Criação e Status dos Deployments:

```bash
kubectl get deployments
```

Saída Esperada:

```
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
backend-deployment   3/3     3            3           25s
```

3. Consultar os ReplicaSets Gerados pelo Deployment:

```bash
kubectl get replicasets
```

(Também aceita o atalho: `kubectl get rs`)

Saída Esperada:

```
NAME                                DESIRED   CURRENT   READY   AGE
backend-deployment-55d84856f6       3         3         3       35s
```

> **Análise Técnica:** *Observe que o nome do ReplicaSet é formado pelo nome do Deployment acrescido do hash de revisão do modelo de Pod (`55d84856f6`).*

4. Consultar os Pods em Execução:

```bash
kubectl get pods -o wide
```

Saída Esperada:

```
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE
backend-deployment-55d84856f6-8x2zp   1/1     Running   0          45s   10.244.1.4   k8s-turma-worker
backend-deployment-55d84856f6-cl9tk   1/1     Running   0          45s   10.244.2.3   k8s-turma-worker2
backend-deployment-55d84856f6-m8pvw   1/1     Running   0          45s   10.244.1.5   k8s-turma-worker
```

5. Verificar o Service e os Endpoints Atribuidos:

```bash
kubectl get service backend-service
kubectl get endpoints backend-service
```

Saída Esperada:

```
NAME              ENDPOINTS                                      AGE
backend-service   10.244.1.4:80,10.244.1.5:80,10.244.2.3:80      1m
```

6. Validar as Variáveis de Ambiente Injetadas dentro do Pod:

Escolha o nome de um dos Pods listados acima e execute o comando `env` interativo:

```
kubectl exec backend-deployment-55d84856f6-8x2zp -- env | grep -E "APP_|LOG_|DB_|WELCOME"
```

Saída Esperada:

```
Plaintext

APP_ENV=production
LOG_LEVEL=info
WELCOME_MSG=Bem-vindo a API orquestrada pelo Kubernetes!
DB_USER=admin_user
DB_PASS=senha_mestra_123
```

> **Sucesso:** *As variáveis vindas do ConfigMap e as variáveis decodificadas do Secret foram injetadas perfeitamente!*

Passo 7: Exercícios de Escalabilidade e Gestão de Rollout

Acompanhe os comandos de escalonamento e gerenciamento do ciclo de vida da aplicação.

1. Escalonar o Deployment Manualmente para 5 Réplicas:

```
Bash

kubectl scale deployment backend-deployment --replicas=5
```

Valide o redimensionamento imediato executando:

```
Bash

kubectl get pods
```

Saída: Você verá 5 Pods no estado `Running`.

2. Simular uma Atualização de Imagem (*Rollout*) para uma Versão Nova:

Alteraremos a imagem do container do Deployment para a versão `nginx:1.26-alpine`:

```bash
kubectl set image deployment/backend-deployment container-backend=nginx:1.26-alpine
```

3. Acompanhar o Status da Atualização Gradual (*Rollout Status*):

```bash
kubectl rollout status deployment/backend-deployment
```

Saída Esperada:

```
Waiting for deployment "backend-deployment" rollout to finish: 2 of 5 updated replicas are available...
Waiting for deployment "backend-deployment" rollout to finish: 4 of 5 updated replicas are available...
deployment "backend-deployment" successfully rolled out
```

4. Consultar o Histórico de Revisões (*Rollout History*):

```bash
kubectl rollout history deployment/backend-deployment
```

Saída Esperada:

```
Plaintext

REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

5. Simular uma Falha de Imagem e Executar a Reversão (*Rollout Undo*):

Vamos intencionalmente atualizar a aplicação informando uma imagem que **não existe** para provocar um erro:

```
bash
kubectl set image deployment/backend-deployment container-backend=nginx:imagem-inexistente-123
```

Acompanhe os Pods falhando com o erro `ErrImagePull` / `ImagePullBackOff`:

```
Bash

kubectl get pods
```

Diante da falha detectada, execute o **Undo** para reverter instantaneamente o cluster para a versão estável anterior (Revisão 2):

```
Bash

kubectl rollout undo deployment/backend-deployment
```

Valide que os Pods foram restaurados com sucesso para o estado saudável:

```bash
kubectl rollout status deployment/backend-deployment
kubectl get pods
```

MÓDULO 6: GUIA DE CONSULTA RÁPIDA (CHEATSHEET DE COMANDOS)

Tabela Resumo de Operações de Cargas de Trabalho e Configurações

Categoria	Comando `kubectl`	Descrição e Finalidade
Aplicações	`kubectl apply -f <arquivo.yaml>`	Cria ou atualiza recursos declarativamente.
Deployments	`kubectl get deployments`	Lista todos os Deployments no namespace atual.
Deployments	`kubectl scale deployment <nome> --replicas=N`	Altera a quantidade de réplicas de um Deployment.
Deployments	`kubectl set image deployment/<nome> <container>=<imagem>`	Atualiza a imagem de um container no Deployment.
Rollouts	`kubectl rollout status deployment/<nome>`	Exibe o progresso em tempo real da atualização.
Rollouts	`kubectl rollout history deployment/<nome>`	Exibe a lista e histórico de revisões do Deployment.
Rollouts	`kubectl rollout undo deployment/<nome>`	Reverte a aplicação para a revisão anterior.
ReplicaSets	`kubectl get replicasets`	Lista os ReplicaSets ativos e o número de réplicas.

Services	`kubectl get services`	Lista os Services, IPs virtuais e portas expostas.
Services	`kubectl get endpoints <nome-servico>`	Exibe os IPs dos Pods associados ao Service.
ConfigMaps	`kubectl get configmaps`	Lista os ConfigMaps gravados no cluster.
ConfigMaps	`kubectl describe configmap <nome>`	Exibe as chaves e conteúdos de um ConfigMap.
Secrets	`kubectl get secrets`	Lista os objetos Secrets no namespace.
Secrets	`kubectl describe secret <nome>`	Exibe os metadados das chaves do Secret (valores velados).

REFERÊNCIAS

Livros e Obras de Referência

BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey; EVENSON, Lachlan. *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. 3. ed. Sebastopol: O'Reilly Media, 2022.

IBRYAM, Bilgin; HUß, Roland. *Kubernetes Patterns: Reusable Elements for Designing Cloud-Native Applications*. 2. ed. Sebastopol: O'Reilly Media, 2023.

HIGHTOWER, Kelsey; BURNS, Brendan; BEDA, Joe. *Kubernetes: Up and Running*. 2. ed. Sebastopol: O'Reilly Media, 2019.

LUKSA, Marko. *Kubernetes in Action*. 2. ed. Manning Publications, 2023.

MAYOR, Michael; HERNANDEZ, Loris. *Kubernetes Security and Observability*. Packt Publishing, 2022.

PULTORAK, David. *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.

SAYFAN, Gigi. *Mastering Kubernetes*. 4. ed. Packt Publishing, 2023.

Recursos Web e Documentação Oficial

CNCF — CLOUD NATIVE COMPUTING FOUNDATION. *Cloud Native Computing Foundation*. Disponível em: https://www.cncf.io/ . Acesso em: 11 out. 2026.

HELM. *Helm Documentation: The Package Manager for Kubernetes*. Disponível em: https://helm.sh/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Kubernetes Documentation*. Disponível em: https://kubernetes.io/docs/ . Acesso em: 11 out. 2026.

KUBERNETES. *Concepts: Cluster Architecture, Workloads, Networking, Storage and Security*. Disponível em: https://kubernetes.io/docs/concepts/ . Acesso em: 11 out. 2026.

KUBERNETES. *Tasks: Tutorials and Administrative Procedures*. Disponível em: https://kubernetes.io/docs/tasks/ . Acesso em: 11 out. 2026.

KUBERNETES. *kubectl Reference Documentation*. Disponível em: https://kubernetes.io/docs/reference/kubectl/ . Acesso em: 11 out. 2026.

MINIKUBE. *Minikube Documentation*. Disponível em: https://minikube.sigs.k8s.io/docs/ . Acesso em: 11 out. 2026.

PROMETHEUS. *Prometheus Documentation*. Disponível em: https://prometheus.io/docs/ . Acesso em: 11 out. 2026.

Referências Complementares para DevOps e Cloud Native

ARGO PROJECT. *Argo CD Documentation*. Disponível em: https://argo-cd.readthedocs.io/ . Acesso em: 11 out. 2026.

CNCF. *Cloud Native Landscape*. Disponível em: https://landscape.cncf.io/ . Acesso em: 11 out. 2026.

DOCKER. *Docker Documentation*. Disponível em: https://docs.docker.com/ . Acesso em: 11 out. 2026.

OPEN CONTAINER INITIATIVE (OCI). *Image Specification and Runtime Specification*. Disponível em: https://opencontainers.org/ . Acesso em: 11 out. 2026.

SIGSTORE. *Sigstore Documentation*. Disponível em: https://docs.sigstore.dev/ . Acesso em: 11 out. 2026.